Desafío 2: Fiesta mundial con UdeAWorld en C++

Integrantes del grupo: Clayder Areiza Pino & Arlington Zahir Llerena

CC: 10421450543 & CC: 72 277 711

Email: clayder.areiza@udea.edu.co, zahir.llerena@udea.edu.co

Número telefónico: + 57 300 726 22 82

Profesores: **AUGUSTO ENRIQUE SALAZAR JIMENEZ & KEVIN DAVID MARTINEZ ZAPATA**

Grupo 003. Código: 2570201 – Informática II -

Facultad de Ingeniería en Telecomunicaciones Región – Yarumal & Carmen del Viboral -

Universidad de Antioquia

19 de abril de 2026

Objetivos técnico general

- **Desarrollar un script robusto en C++ bajo el paradigma de Programación Orientada a Objetos (POO) para la simulación automatizada del Mundial de Fútbol FIFA 2026. El sistema debe integrar modelos matemáticos predictivos y estructuras de datos de autoría propia para gestionar eficientemente la información de equipos, jugadores y encuentros, garantizando un control absoluto sobre el uso de recursos computacionales.**

Objetivos específicos

- **Implementar estructuras de datos dinámicas originales mediante el uso de plantillas (templates) y gestión de punteros, sustituyendo completamente los contenedores de la STL y los "structs" para soportar la composición de entidades como equipos y plantillas de jugadores.**
- **Programar un motor de simulación probabilístico fundamentado en la ecuación de esperanza de goles dada en el documento guía. Que automatice el flujo competitivo del torneo, desde el sorteo de grupos con restricciones geográficas hasta la gran final, actualizando estadísticas históricas en tiempo real.**

- **Implementar un módulo de auditoría de recursos que cuantifique la eficiencia del programa mediante el reporte exacto de la memoria dinámica consumida y el conteo de iteraciones ejecutadas en cada funcionalidad, asegurando el cumplimiento de los estándares de optimización exigidos por el profesor.**

Propuesta preliminar desarrollo.

Antes de iniciar cualquier proyecto ingenieril, independiente de la disciplina de profundización. Planteamos unas preguntas que permiten direccionar la propuesta y además nos brinda la oportunidad de esclarecer conceptos vinculados con ejercicios de programación en el lenguaje C++. A continuación, damos a conocer las preguntas que desde nuestra opinión establecen la ruta de trabajo.

Fase 1: Estructuras de Datos y Memoria (Nivel Base)

- **Gestión de Memoria: ¿Cuál será nuestra estrategia de "redimensionamiento manual"? Dado que el uso de la STL (como std::vector) está prohibido, nos preguntamos: ¿Cómo gestionaremos el crecimiento de las colecciones de equipos y jugadores cuando se lleve a cabo la simulación de etapas?**
- **Monitoreo de Recursos: ¿Cómo implementar el "monitoreo de recursos" que sea consecuente con la actualización de información entrante y saliente? El sistema requiere reportar la memoria consumida en todas las etapas cada una de ellas definidas por unas estructuras y variables locales con funcionalidades puntuales. Por el momento, creemos que el diseño de una función acumulada que reporte de manera periódica el consumo de memoria es una solución parcial.**
- **Abstracción vs. Restricción: ¿Cómo sustituir los "structs" sin generar sobrecarga de código y uso eficiente del almacenamiento? La ventaje del uso de la sentencia "structs" es que permitía almacenar información de manera óptima y utilizar los atributos de estos, para la carga posterior de los elementos almacenados. Como alternativa a "structs", diseñaremos clases para que actúen como contenedores de datos ligeros pero manteniendo el encapsulamiento y la abstracción que exige la POO.**

Fase 2: Lógica de Manipulación de data (Nivel Intermedio)

- **¿Cómo procesaremos el CSV utilizando solo aritmética de punteros? Al no poder usar librerías externas para leer el archivo. ¿Cómo implementar una función de "split" manual que identifique las comas y convierta cadenas de caracteres (char*) a tipos numéricos, asegurando que no queden residuos y**

además que no sean leídos por las sentencias dadas en mi script y afecten las salidas en la terminal de C++?

- **Estructura de Datos Propia: ¿Lista Enlazada o Arreglo Dinámico Template?** El documento de este desafío recomienda que las estructuras de datos sean de creación propia y utilicen plantillas (templates). Para acceder al ranking FIFA o buscar goleadores, ¿qué estructura ofrece el menor número de iteraciones, es de aclarar que esto es importante puesto que en un código valorar el tiempo requerido para llevar a cabo iteraciones es una métrica de evaluación fundamental.

Fase 3: Algoritmos de Juego y Optimización (Nivel Avanzado)

- **Modelo de Simulación: ¿Cómo garantizar la convergencia de la probabilidad de goles?** La cantidad de goles esperados se rige por la ecuación:

$$\lambda_A = \mu \cdot \left(\frac{GF_A}{\mu}\right)^{\alpha} \cdot \left(\frac{GC_B}{\mu}\right)^{\beta}$$

  Si cada jugador tiene solo un 4% de probabilidad de anotar, ¿cómo estructuraremos el bucle de simulación para que la suma de goles individuales no exceda o subestime drásticamente el λ_A calculado para el equipo? (No tenemos ni idea de cómo abordar este problema).

- **Modularidad y Enlace: ¿Cómo evitar las dependencias circulares en un sistema multi-archivo?** Al usar una arquitectura de archivos .h y .cpp, surge el reto:
  Si la clase Equipo tiene una Lista<Jugador> y la clase Partido necesita acceder a ambos, ¿cómo organizaremos la estructura de código y el uso de referencias para que el compilador no genere errores de redefinición ni copias innecesarias de datos estructurados?

Aunque las preguntas anteriores, pueden dar la impresión de ser triviales, las respuestas nos permiten dimensionar la complejidad de la iniciativa llamada desafío 2. Así mismo, nos brindan pautas de que elementos son los primarios para realizar una propuesta consecuente con los requerimientos puntuales del reto. Atendiendo a lo comentado por el momento, a continuación, damos a conocer una propuesta general que hemos pensado para la solución del reto. Por supuesto, es una propuesta preliminar que probablemente cambien en la hora de llevar a cabo la construcción de los algoritmos por módulos.

1. **Propuesta preliminar que da a conocer un enfoque de como inicialmente planteamos el problema.**

Esta es la hoja de ruta para el desarrollo de UdeAWorldCup. Al tratarse de un sistema con restricciones de bajo nivel (sin STL y sin struct), la construcción debe ser ascendente: primero fabricamos las herramientas y luego el simulador.

- **Etapa 1: El Cimiento Técnico (Estructuras Propias): Dado que no se pueden usar contenedores estándar, esta etapa consiste en crear nuestras propias funciones de gestión de datos.**
  - **Detalle: Implementar una clase Plantilla (Template) llamada Lista<T>. Esta clase debe usar un puntero T* para reservar espacio en la memoria dinámica (new) y un método para redimensionar el arreglo si se llena.**
  - **Por qué es vital: Sin esto, no tendremos dónde guardar a los 48 equipos ni a los jugadores.**
  - **Requisito de Eficiencia: Aquí es donde decidimos si el acceso será por índice o por búsqueda secuencial.**

- **Etapa 2: El Modelo de Datos (Entidades POO): Aquí transformamos los conceptos del fútbol en objetos de C++ utilizando abstracción y encapsulamiento.**
  - **Clase Jugador: Atributos como nombre, número y sus estadísticas personales (goles, tarjetas, faltas).**
  - **Clase Equipo: Contiene el ranking FIFA, la confederación y una instancia de tu Lista<Jugador>.**
  - **Clase Partido: Almacena los dos equipos, la sede y el resultado final.**

- **Etapa 3: El Gestor de Información (Valoración entrada salida I/O): Esta etapa conecta el mundo exterior (archivos) con la memoria de nuestros respectivos programa.**
  - **Carga de Datos: Debemos programar un lector manual que procese el archivo .csv. Como no se permite ANSI C, usaremos punteros y bucles para identificar las comas y separar los nombres de los números.**
  - **Generación de Reportes: Crear funciones que exporten los resultados de cada etapa a archivos .txt para que el "cliente" pueda revisar el progreso.**

- **Etapa 4: El Cerebro Matemático: Es el núcleo lógico donde se aplican las fórmulas y las reglas del torneo.**
  - **Motor de Goles: Implementación de la ecuación de probabilidad para λ_A:**

$$\lambda_A = \mu \cdot \left(\frac{GF_A}{\mu}\right)^{\alpha} \cdot \left(\frac{GC_B}{\mu}\right)^{\beta}$$

- **Gestor del Torneo: Una clase que controle el flujo: desde el sorteo de grupos (evitando que selecciones de la misma confederación coincidan) hasta la gran final.**

- **Etapa 5: El Auditor de Recursos (Eficiencia): Esta etapa es transversal porque pretendemos cuantificar qué tan "pesado" y "lento" seria nuestro código.**
    - **Métrica de Memoria: Una función que sume los bytes ocupados por todos los objetos creados y las variables locales en uso.**
    - **Métrica de Iteraciones: Un contador que registre cuántas veces el procesador repitió una tarea para completar una funcionalidad (ej. cuántos ciclos tomó ordenar la tabla de posiciones).**

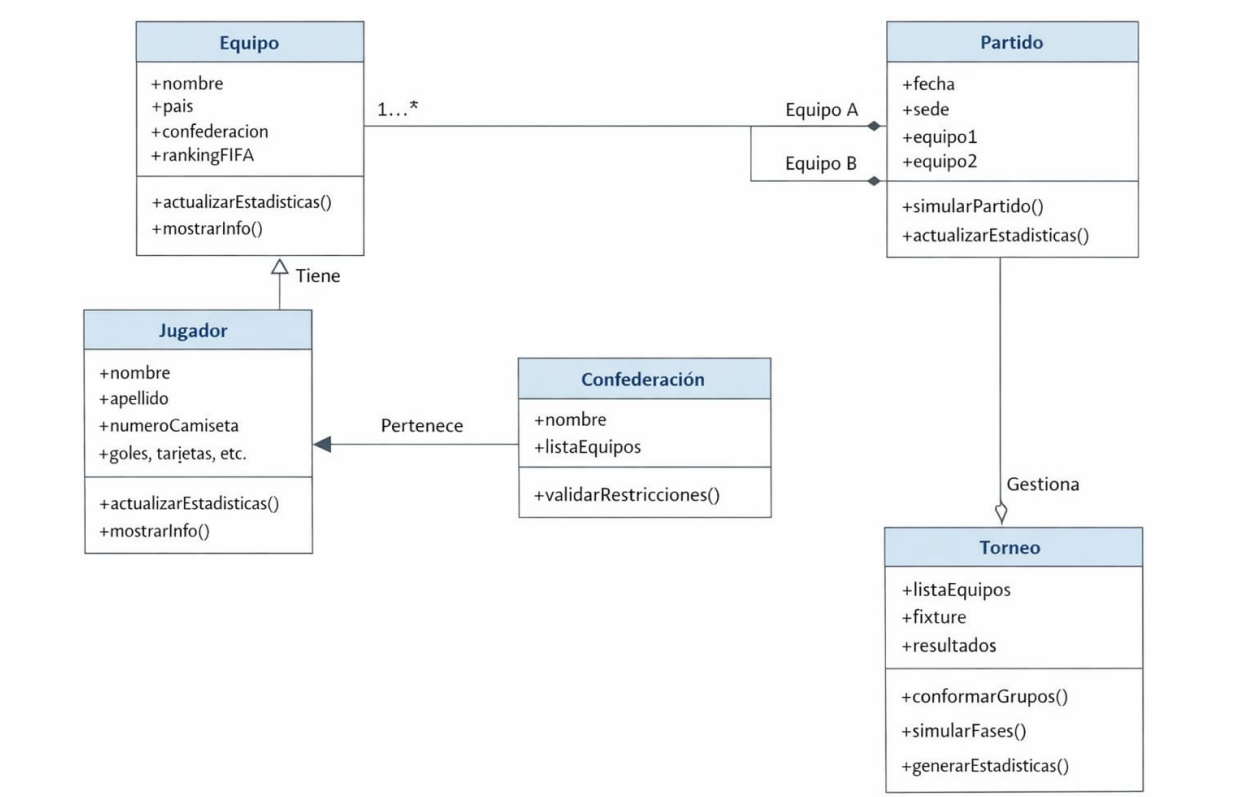
Interconexión:

La interconexión entre las distintas funciones, clases definidas pretendemos establecerlas por las etapas anteriormente descritas. Básicamente visualizamos la iniciativa como una línea de ensamblaje industrial:

1. **Entrada: La Etapa 3 lee el archivo físico. Por cada línea, crea un objeto de la Etapa 2.**
2. **Almacenamiento: Estos objetos se guardan dentro de la estructura diseñada en la Etapa 1. La Etapa 5 empieza a contar cuánta memoria se usó en este proceso.**
3. **Procesamiento: El simulador de la Etapa 4 toma los equipos de las listas, calcula los goles con la fórmula matemática y actualiza las estadísticas de los objetos de la Etapa 2.**
4. **Salida: La Etapa 3 vuelve a activarse para escribir el reporte final en un .txt, mientras la Etapa 5 muestra en pantalla el costo total de la ejecución.**

Esta estructura asegura (o por lo menos lo creemos así) que, si decidimos cambiar la fórmula de goles (Etapa 4), no tengamos que reescribir cómo se leen los archivos (Etapa 3) ni cómo se guardan en memoria (Etapa 1).

El diagrama a continuación da a conocer las posibles asociaciones que debemos tener en cuenta para la construcción de la propuesta de manera modular, atendiendo a las etapas que definimos previamente y dando respuesta a las preguntas del inicio del documento.



Conclusiones preliminares.

La fase de planificación ha permitido transformar un documento de requisitos complejo en una hoja de ruta técnica estructurada, garantizando que la solución final no sea solo funcional, sino también eficiente bajo las estrictas restricciones dada por el profesor de Teoría de Informática II de la Facultad de Ingeniería de la de la Universidad de Antioquia.

Los pilares de lo alcanzado hasta hoy son:

- **Dominio de las Restricciones Técnicas: Se ha interiorizado la prohibición de la STL, los "structs" y el lenguaje ANSI C. Esto nos obliga a una implementación pura en C++ basada en clases, plantillas y gestión manual de memoria dinámica.**

- ◊ **Arquitectura Modular Definida: Mediante el diseño por etapas, hemos establecido un flujo lógico que comienza con la creación de herramientas de almacenamiento propias (templates) y culmina en un motor de simulación interconectado.**
- ◊ **Fundamentación Cuantitativa: El núcleo del programa ya cuenta con la validación de la ecuación de esperanza de goles, asegurando que la simulación de los 104 partidos sea estadísticamente coherente con el ranking FIFA y el histórico de cada equipo.**
- ◊ **Alineación con los Criterios de Evaluación: Los objetivos general y específicos propuestos cubren los tres ejes fundamentales del desafío: la correcta aplicación de la POO, la eficiencia computacional (conteo de iteraciones) y la transparencia en el consumo de memoria.**

Bibliografía.

- **Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley. (2010).**
- **Deitel, P. J., & Deitel, H. M. (2017). *C++ How to Program* (10th ed.). Pearson.**
- **Prata, S. (2011). *C++ Primer Plus* (6th ed.). Addison-Wesley.**